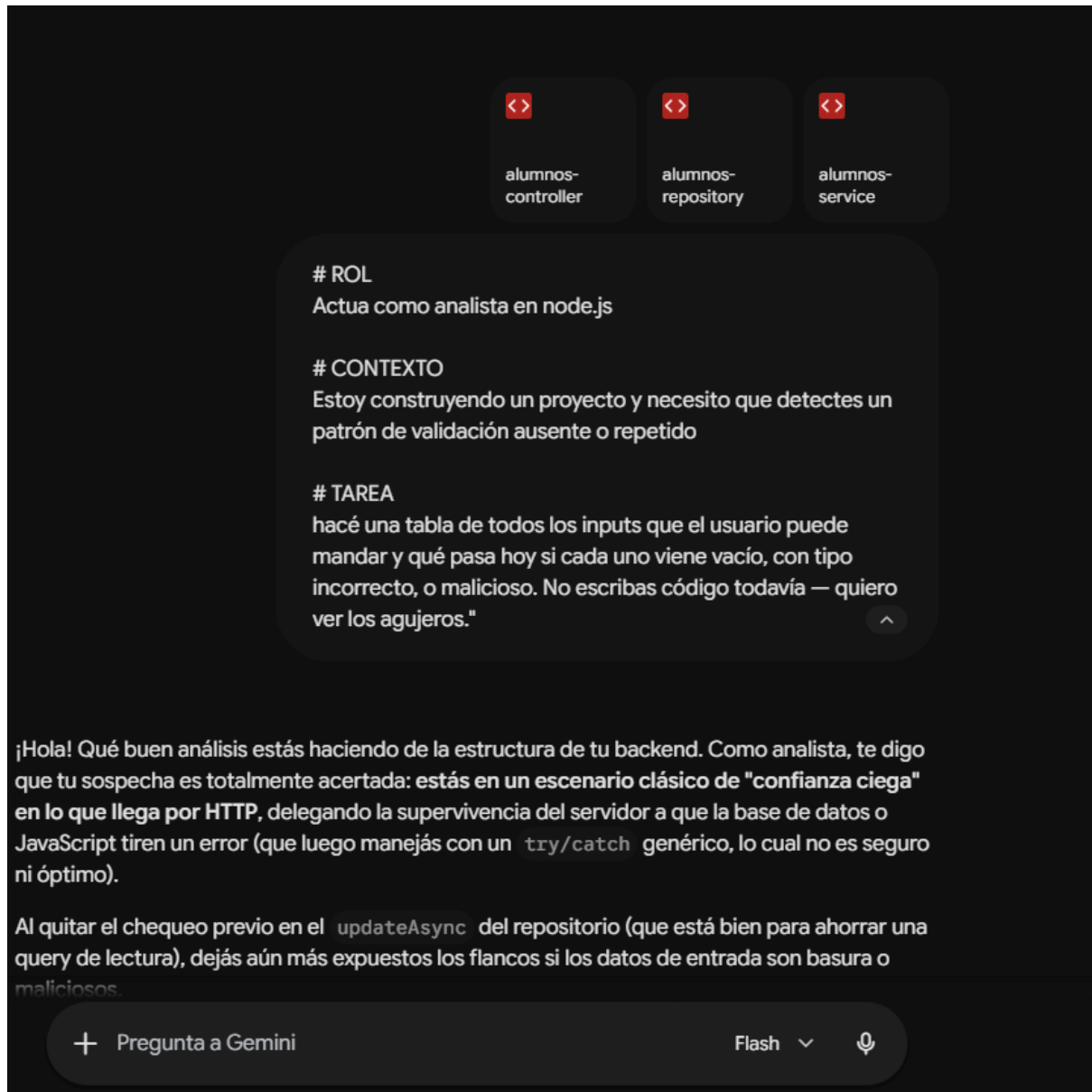


PROMPT 1, VALIDACIÓN:



RESPUESTA VALIDACION:

¡Hola! Qué buen análisis estás haciendo de la estructura de tu backend. Como analista, te digo que tu sospecha es totalmente acertada: **estás en un escenario clásico de "confianza ciega" en lo que llega por HTTP**, delegando la supervivencia del servidor a que la base de datos o JavaScript tiren un error (que luego manejas con un `try/catch` genérico, lo cual no es seguro ni óptimo).

Al quitar el chequeo previo en el `updateAsync` del repositorio (que está bien para ahorrar una query de lectura), dejás aún más expuestos los flancos si los datos de entrada son basura o maliciosos.

A continuación, prepararé el mapa de calor con **todos los "agujeros" (vulnerabilidades, bugs y comportamientos inesperados)** de tus endpoints actuales sin escribir una sola línea de código de validación.

Análisis de Impacto de Inputs (Agujeros de Validación)

Endpoint	Input / Origen	Si viene vacío (null / undefined / "")	Si viene con tipo incorrecto (ej: string por int, o arrays)	Comportamiento Malicioso (Inyección, manipulación de lógica, etc.)
GET /:id	req.params.id	N/A (La ruta no matchearía si está vacío, iría a GET /).	Se pasa el string tal cual a la DB. Si es un texto (ej: 'abc'), la base de datos Postgres lanzará un error de casteo (invalid input syntax for integer) rompiendo la query. Retorna 500 Internal Server Error.	SQL Injection Indirecto: Aunque dependés de cómo esté implementado GenericRepository, si concatena strings en lugar de usar placeholders (\$1), un ID malicioso compromete toda la DB.
DELETE /:id	req.params.id	N/A (Sigue el mismo comportamiento de ruteo).	Mismo comportamiento que GET /:id. Explota en la base de datos por tipo de dato inválido y devuelve un 500.	SQL Injection / Escalación: Si el ID es una query, puede causar estragos según la composición del repositorio genérico.
POST /	req.body completo	Si mandás un body vacío ({}),	Si mandás tipos cruzados	Polución de Prototipos /

		el backend creará un alumno vacío. El repositorio usa coalescencia (?? "", ?? 0), por lo que insertará campos por defecto ("", 0, 1970-01-01 o null).	(ej: id_curso: "no-soy-un-numero"), explota la query SQL de inserción o la query de validación del curso en el Service, retornando 400 Bad Request.	Overposting: Podés inyectar propiedades inesperadas en el objeto entity que viaja directo al Service y Repo. Aunque el SQL tiene columnas fijas, si un middleware posterior lee el body, puede ser explotado.
POST /	entity.id_curso	Al venir vacío, la validación del curso en el Service hace un <i>early return</i> (if (!idCurso) return), por lo que permite guardar un alumno sin curso (o con curso 0 por el default del Repo).	Si se envía un string o un objeto, la query de búsqueda de curso fallará estrepitosamente en CursosService.getByIdAsync.	Se pueden enviar IDs negativos o flotantes que podrían romper la lógica de negocio de cursos.
POST /	entity.fecha_nacimiento	Si no viene, el Repo guarda null. Sin embargo, al listarlo, calcularEdad recibirá null y el get devolverá la entidad sin romper, pero sin	Si se envía un string que no es fecha (ej: 'no-soy-fecha'), new Date('no-soy-fecha') crea un objeto Invalid Date. Al restar los años en calcularEdad,	DoS (Denegación de Servicio) por CPU / Desborde: Formatos de fechas extremadamente grandes o inválidos que bloqueen el

		consistencia si la fecha de nacimiento es obligatoria en tu negocio.	retornará NaN como edad del alumno.	event loop en el parsing.
PUT /:id	req.params.id	N/A (Ruta).	Se parsea con <code>parseInt(req.params.id)</code> . Si es un string inválido (ej: 'abc'), id se convierte en NaN.	Si id es NaN, se le asigna <code>entity.id = NaN</code> y se envía al Repo. La query de <code>UPDATE ... WHERE id = \$1</code> fallará o simplemente afectará 0 filas, devolviendo un confuso 404 Not Found en lugar de un 400.
PUT /:id	req.body completo	Si viene un body vacío (<code>{}</code>), el repo actualizará absolutamente todos los campos del alumno a sus valores por defecto ("", 0, null), pisando y destruyendo la información preexistente.	Mismo comportamiento de parseo e inconsistencias que en el POST.	Manipulación de Datos Descontrolada: No hay protección de campos de "solo lectura". Si el body trae propiedades del sistema, se procesan sin filtro.



Diagnóstico de los "Agujeros" Clave

1. La "Edad" como un peligro silencioso (NaN):

Si la fecha de nacimiento que viene de la base de datos está corrupta o se insertó un

string inválido en un bypass anterior, la función helper calcularEdad generará un NaN. Al serializar a JSON para responder al cliente, NaN se transforma en null de forma silenciosa, ocultando el error de casteo en el transporte pero rompiendo lógicas del lado del cliente.

2. **Cero Sanitización de Strings (nombre / apellido):**

No hay límites de caracteres. Un usuario podría mandar un nombre de 50,000 caracteres como payload y saturar la memoria o desbordar el tamaño de almacenamiento en Postgres. Tampoco hay limpieza de tags HTML/JS (vulnerable a **Stored XSS** si estos nombres se renderizan luego en un frontend).

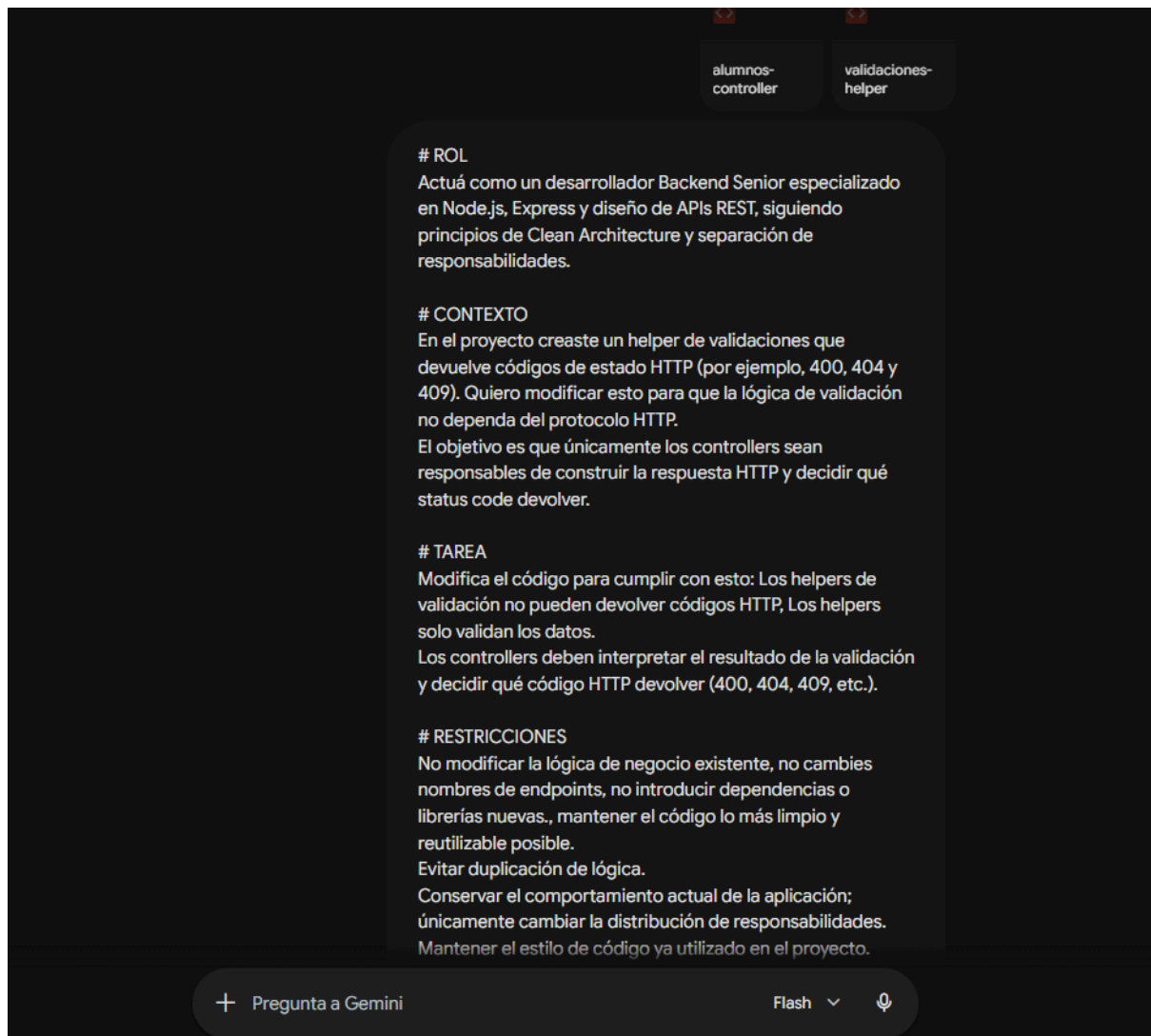
3. **El peligro del valor por defecto en Base de Datos (?? en Repositorio):**

El uso de entity?.nombre ?? " en el repositorio es un antipatrón de persistencia si el campo en la DB es requerido (NOT NULL). Estás enmascarando un dato faltante (un payload inválido) guardando un string vacío en su lugar, cuando el comportamiento correcto debería ser rechazar la petición en la puerta de entrada (el controlador).

Próximo Paso

¿Querés que definamos la estrategia para solucionar esto? Podemos ir por el camino de diseñar **Middlewares de Validación** con alguna librería estándar de Node (como Joi o Zod) para frenar toda esta basura en la capa del Router antes de que siquiera toque tus Services. ¿Cómo preferís que lo encaremos?

PROMPT 2:



1. El Helper de Validaciones (src/helpers/validaciones-helper.js)

Agregamos un tercer middleware en el helper que se encarga de la validación de negocio (la existencia del curso en la DB). Al ser un middleware, tiene acceso a req y res, y puede consultar al servicio correspondiente.

JavaScript

```
import { StatusCodes } from 'http-status-codes';  
import ResponseHelper from '../response-helper.js';  
import CursosService from '../services/cursos-service.js'; // Importamos el servicio aquí
```

```
const cursosService = new CursosService();
```

```
export default class ValidacionesHelper {
```

```

/**
 * Middleware 1: Valida y parsea el ID de los parámetros de la ruta
 */
static validarParamIdMiddleware(req, res, next) {
  const idOriginal = req.params.id;

  // Validar formato (solo dígitos)
  const esEnteroValido = /^[d]+$/.test(idOriginal);
  if (!esEnteroValido) {
    return ResponseHelper.json(res, StatusCodes.BAD_REQUEST, {
      error: 'Input inválido',
      detalle: `El parámetro ID proporcionado ('${idOriginal}') debe ser un número entero positivo.`
    });
  }

  const idParseado = parseInt(idOriginal, 10);
  if (idParseado <= 0) {
    return ResponseHelper.json(res, StatusCodes.BAD_REQUEST, {
      error: 'Input inválido',
      detalle: 'El parámetro ID debe ser mayor a cero.'
    });
  }

  // Dejamos el ID limpio para el controlador
  req.params.idClean = idParseado;
  next();
}

/**
 * Middleware 2: Valida la estructura y tipos de datos del Alumno (Body)
 */
static validarAlumnoBodyMiddleware(req, res, next) {
  const body = req.body;
  const errores = [];

  if (body.nombre === undefined || body.nombre === null || String(body.nombre).trim() === "") {
    errores.push("El campo 'nombre' es requerido y no puede estar vacío.");
  } else if (typeof body.nombre !== 'string' || body.nombre.length > 100) {
    errores.push("El campo 'nombre' debe ser un texto de máximo 100 caracteres.");
  }

  if (body.apellido === undefined || body.apellido === null || String(body.apellido).trim() === "") {
    errores.push("El campo 'apellido' es requerido y no puede estar vacío.");
  } else if (typeof body.apellido !== 'string' || body.apellido.length > 100) {
    errores.push("El campo 'apellido' debe ser un texto de máximo 100 caracteres.");
  }
}

```

```

    }

    if (body.id_curso !== undefined && body.id_curso !== null) {
        const esEntero = Number.isInteger(body.id_curso) ||
/^\\d+$/\\.test(String(body.id_curso));
        if (!esEntero || parseInt(body.id_curso, 10) <= 0) {
            errores.push("El campo 'id_curso' debe ser un número entero positivo válido.");
        }
    } else {
        errores.push("El campo 'id_curso' es requerido.");
    }

    if (body.fecha_nacimiento) {
        const esFechaValida = !isNaN(Date.parse(body.fecha_nacimiento));
        if (!esFechaValida) {
            errores.push("El campo 'fecha_nacimiento' debe ser una fecha válida (formato
recomendado: YYYY-MM-DD).");
        }
    }

    if (body.hace_deportes !== undefined && body.hace_deportes !== null) {
        if (typeof body.hace_deportes !== 'boolean') {
            errores.push("El campo 'hace_deportes' debe ser un valor booleano (true o
false).");
        }
    }

    if (errores.length > 0) {
        return ResponseHelper.json(res, StatusCodes.BAD_REQUEST, {
            error: 'Input inválido',
            detalles: errores
        });
    }

    next();
}

/**
 * Middleware 3: Valida que el curso asociado realmente exista en la Base de Datos
(Negocio)
 */
static async validarCursoExisteMiddleware(req, res, next) {
    const idCurso = req.body.id_curso;

    if (!idCurso) {
        return next(); // Si no hay id_curso (y pasó el middleware anterior), seguimos de largo
    }

```



```

    try {
      const curso = await cursosService.getByIdAsync(idCurso);
      if (curso == null) {
        // Devolvemos 409 (Conflicto) porque viola la integridad referencial de tu negocio
        return ResponseHelper.json(res, StatusCodes.CONFLICT, {
          error: 'Conflicto',
          detalle: `El curso con id ${idCurso} no existe en el sistema.`
        });
      }
      next(); // El curso existe, podemos continuar con el siguiente paso
    } catch (error) {
      console.error('Error al validar existencia del curso en middleware:', error);
      return ResponseHelper.json(res, StatusCodes.INTERNAL_SERVER_ERROR, {
        error: 'Error inesperado',
        detalle: 'No se pudo verificar la existencia del curso.'
      });
    }
  }
}
}
}

```

2. Cómo se aplican en las Rutas (src/controllers/alumnos-controller.js)

Ahora las rutas encadenan secuencialmente los middlewares de validación. El controlador queda 100% libre de lógica de verificación y asume que todo lo que entra es impecable.

```

JavaScript
// ... Imports idénticos ...

// POST /
// Encadena: Validar Estructura -> Validar Existencia del Curso -> Ejecutar Controlador
router.post("",
  ValidacionesHelper.validarAlumnoBodyMiddleware,
  ValidacionesHelper.validarCursoExisteMiddleware,
  async (req, res) => {
    try {
      let entity = req.body;
      const newId = await currentService.createAsync(entity);
      if (newId > 0 ){
        return ResponseHelper.json(res, StatusCodes.CREATED, { id: newId });
      } else {
        return ResponseHelper.json(res, StatusCodes.BAD_REQUEST, { error: 'Input
        inválido' });
      }
    }
  }
)

```

```

    }
  } catch (error) {
    console.error(error);
    return ResponseHelper.json(res, StatusCodes.INTERNAL_SERVER_ERROR, {
      error: 'Error inesperado' });
  }
}
);

```

```

// PUT /:id
// Encadena: Validar ID de ruta -> Validar Body -> Validar Existencia del Curso -> Ejecutar
Controlador

```

```

router.put('/:id',
  ValidacionesHelper.validarParamIdMiddleware,
  ValidacionesHelper.validarAlumnoBodyMiddleware,
  ValidacionesHelper.validarCursoExisteMiddleware,
  async (req, res) => {
    try {
      let id = req.params.idClean;
      let entity = req.body;

```

```

      if (entity.id && parseInt(entity.id, 10) !== id) {
        return ResponseHelper.json(res, StatusCodes.BAD_REQUEST, {
          error: 'Input inválido',
          detalle: `El ID de la URL (${id}) no coincide con el ID del cuerpo (${entity.id}).`
        });
      }
    }
  }

```

```

    entity.id = id;
    const rowsAffected = await currentService.updateAsync(entity);

```

```

    if (rowsAffected !== 0){
      return ResponseHelper.json(res, StatusCodes.OK, { message: 'Registro
actualizado con éxito.', rowsAffected });
    } else {

```

```

      return ResponseHelper.json(res, StatusCodes.NOT_FOUND, {
        error: 'No encontrado',
        detalle: `No se encontró la entidad de Alumno con ID: ${id}`
      });
    }
  } catch (error) {
    console.error(error);
    return ResponseHelper.json(res, StatusCodes.INTERNAL_SERVER_ERROR, {
      error: 'Error inesperado' });
  }
}
);

```

3. Simplificación del Servicio (src/services/alumnos-service.js)

Con este cambio, podés **limpiar por completo** el servicio. Ya no necesita instanciar CursosService, ni tener métodos como validarCursoExiste, reduciendo el acoplamiento drásticamente:

JavaScript

```
import AlumnosRepository from '../repositories/alumnos-repository.js';  
// ¡Ya no es necesario importar CursosService aquí!
```

```
function calcularEdad(fechaNacimiento) {  
  if (!fechaNacimiento) return null;  
  const hoy = new Date();  
  const nacimiento = new Date(fechaNacimiento);  
  let edad = hoy.getFullYear() - nacimiento.getFullYear();  
  const mesDiff = hoy.getMonth() - nacimiento.getMonth();  
  if (mesDiff < 0 || (mesDiff === 0 && hoy.getDate() < nacimiento.getDate())) {  
    edad--;  
  }  
  return edad;  
}
```

```
function agregarEdad(alumno) {  
  if (!alumno) return alumno;  
  return { ...alumno, edad: calcularEdad(alumno.fecha_nacimiento) };  
}
```

```
export default class AlumnosService {  
  constructor() {  
    this.AlumnosRepository = new AlumnosRepository();  
  }
```

```
  getAllAsync = async () => {  
    const returnArray = await this.AlumnosRepository.getAllAsync();  
    if (returnArray === null) return null;  
    return returnArray.map(alumno => agregarEdad(alumno));  
  }
```

```
  getByIdAsync = async (id) => {  
    const returnEntity = await this.AlumnosRepository.getByIdAsync(id);  
    return agregarEdad(returnEntity);  
  }
```

```

createAsync = async (entity) => {
  // Directo al repositorio, el middleware ya garantizó que todo es válido y el curso existe
  return await this.AlumnosRepository.createAsync(entity);
}

updateAsync = async (entity) => {
  // Directo al repositorio
  return await this.AlumnosRepository.updateAsync(entity);
}

deleteByIdAsync = async (id) => {
  return await this.AlumnosRepository.deleteByIdAsync(id);
}
}

```

PROMPT 3

<>

alumnos-controller

<>

validaciones-helper

ROL
Actúa como un desarrollador Backend Senior especializado en Node.js, Express y diseño de APIs REST, siguiendo principios de Clean Architecture y separación de responsabilidades.

CONTEXTO
En el proyecto creaste un helper de validaciones que devuelve códigos de estado HTTP (por ejemplo, 400, 404 y 409). Quiero modificar esto para que la lógica de validación no dependa del protocolo HTTP.
El objetivo es que únicamente los controllers sean responsables de construir la respuesta HTTP y decidir qué status code devolver.

TAREA
Modifica el código para cumplir con esto: Los helpers de validación no pueden devolver códigos HTTP, Los helpers solo validan los datos.
Los controllers deben interpretar el resultado de la validación y decidir qué código HTTP devolver (400, 404, 409, etc.).

RESTRICCIONES
No modificar la lógica de negocio existente, no cambies nombres de endpoints, no introducir dependencias o librerías nuevas., mantener el código lo más limpio y reutilizable posible.
Evitar duplicación de lógica.
Conservar el comportamiento actual de la aplicación; únicamente cambiar la distribución de responsabilidades.

+

Pregunta a Gemini

Flash

🗣️